AI Travel Planner: Design Report

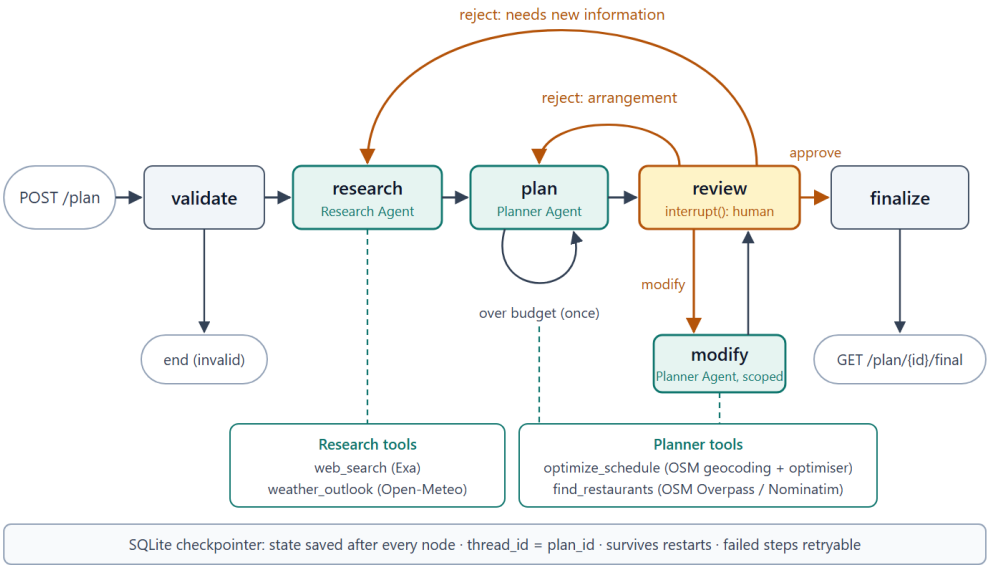
1. What it does

I built a trip planner with LangGraph and FastAPI. It researches a destination, drafts an itinerary and then waits for a person to review it.

You send a destination, dates, a budget range, a currency, interests, a traveller count and notes. A research agent gathers information and a planner agent turns it into a daily itinerary. The reviewer can approve the draft, reject it with feedback, or ask for changes to named days.

There are 2 agents, 4 tools, 45 offline tests and 4 opt-in live checks. A saved Lisbon session goes through all 3 review actions (section 7).

2. How it works



1. `POST /plan` sends the request. The graph in `app/graph.py` validates it. A bad request stops with the status `invalid`.
2. The research agent searches the web and checks the weather. It returns a `ResearchReport`.
3. The planner agent returns an `Itinerary` (lodging, meals and timed visits).
4. Python adds up the group cost and labels the draft `under`, `within` or `over` the budget. A draft over `budget_max` gets 1 automatic revision.
5. The `review` node pauses the graph until a person approves, rejects or modifies the draft.
6. On approval, `finalize` builds the final response, which you read at `GET /plan/{id}/final`.

3. The agents and tools

Both agents live in `app/agents.py`. They use LangChain's `create_agent` loop and return Pydantic objects. The research agent has the first two tools. The planner agent has the last two.

Tool	What it does	Data source
`web_search`	Returns 5 results with title, URL, date and highlights	Exa (API key needed)
`weather_outlook`	Forecast if the whole trip is within 16 days, otherwise the same dates a year earlier, labelled historical	Open-Meteo (no key)
`optimize_schedule`	Groups stops into days, orders them, returns timed visits and warnings	OpenStreetMap geocoding and a built-in optimiser (no key)
`find_restaurants`	Finds places in a radius, sorted by distance	OpenStreetMap Overpass, then Nominatim as a last fallback (no key)

The restaurant tool relaxes the cuisine filter if nothing matches, but never the vegetarian filter.

4. Human review

The `review` node stops at `interrupt()`, and `POST /plan/{id}/review` resumes it. The default limit is 5 reject or modify rounds, then the API accepts approval only.

Action	What happens
approve	`finalize` copies the draft unchanged into the `itinerary` field and adds the request, research, timestamps and review history. No extra model call.
reject	The reviewer gives feedback and can set `revise` to `research` or `itinerary`. Otherwise `route_feedback()` picks, and defaults to the planner if that call fails.
modify	The reviewer names the days to change. After the planner replies, `keep_unchanged()` restores every other day, the lodging and the top-level text.

A change without a day (for example the hotel) can update the lodging and the summary text. Every day stays exactly as it was.

5. Saving state and recovery

A checkpoint is a saved copy of the plan's state. The SQLite checkpointer writes one after every node, with `thread_id` set to the `plan_id`.

The runner uses `durability="sync"`, so each checkpoint is written before the next step starts. A failed step resumes with `POST /plan/{id}/retry`. On startup the runner resumes stalled plans. Failed plans wait for a retry, and drafts awaiting review stay paused.

Status	Meaning
`queued`	Accepted, not started
`validating`, `researching`, `planning`, `modifying`, `finalizing`	A step is running
`processing`	Between checkpoint updates
`awaiting_review`	Paused for a person
`completed`	The graph finished
`failed`	A step raised an error
`stalled`	The run lost its active task
`invalid`	The request was rejected

6. API

Model runs can take minutes, so the API replies at once and a background runner does the work.

Method	Path	Purpose
POST	`/plan`	Submit a request and start the workflow
GET	`/plan/{id}`	Status, draft, research, reviews and progress events
POST	`/plan/{id}/review`	Approve, reject (with feedback) or modify (with changes)
GET	`/plan/{id}/final`	The approved plan
GET	`/plan/{id}/events`	Server-sent events, one per status change
POST	`/plan/{id}/retry`	Resume a failed or stalled run from its checkpoint

Codes: `202` accepted, `404` unknown plan, `409` wrong state or round limit reached, `422` bad input. Validation rejects past or reversed dates, trips over 14 days, reversed budget bounds, empty interests and unknown modification days. The currency must be 3 uppercase letters, with no ISO list check.

7. A real run: Lisbon

This is a saved session from 22 October 2026. The request was for Lisbon, 20-22 November, for 2 travellers with a group budget of 900-1600 EUR. Interests were food, history and viewpoints. One traveller was vegetarian.

Step	Result
Research	6 Exa searches and 1 weather call gave 10 attractions and 9 sources
Draft 1	Cost 897 EUR, below the 900 minimum, so marked `under`
Reject	I asked for less public transport, one area per day and a fado evening. The router sent it to research. Draft 2 cost 1417 EUR
Modify day 2	I asked for a sunset viewpoint and a vegetarian dinner nearby. Draft 3 put Miradouro de Santa Luzia at 16:30 and cost 1221 EUR. Days 1 and 3 and the lodging matched draft 2
Approve	Revision 3 became the `itinerary` of the final response

Three things were not good:

- Drafts 2 and 3 came from `gemini-3.5-flash-lite`, because the free-tier quota ran out during the session. The output is weaker for it.
- Draft 2 scheduled fado on all 3 evenings when I had asked for one. Draft 3 kept the fado evenings on days 1 and 3.
- For the day 2 vegetarian dinner, the planner searched with cuisine `vegetarian` but without the `vegetarian=True` filter. It chose A Muralha Tasca Tipica. The saved record does not confirm a vegetarian menu.

8. Testing and what broke

On 26 October 2026, `pytest -m "not live"` passed all 45 offline tests. The graph and API tests use fake agents. The wiring test uses the real agent loop with a scripted model.

File	Tests	Covers
`test_graph.py`	10	Review routes, budget retry, saved state, retry
`test_api.py`	13	HTTP responses, round limit, SSE
`test_tools.py`	19	Scheduling, weather, restaurant fallbacks, Exa, budget maths
`test_runner.py`	2	Plan status while a step is active
`test_agent_wiring.py`	1	Real agents inside the graph
`test_live.py`	4 (opt-in)	Open-Meteo, Nominatim, Overpass, Exa. Needs `RUN_LIVE=1`

The live run exposed four problems.

- **Status race.** An active plan could briefly read `invalid` between a node's writes and its checkpoint. The runner now checks its task registry first.
- **Quota and overload.** I saw Gemini `503` responses, a quota limit and Overpass `502` responses. I added model fallbacks and a restaurant retry, mirror and Nominatim sequence. A fallback can still give a worse draft.
- **Runaway search.** One research pass made 29 web searches. Each agent run is now capped at 8 web searches, 1 weather call, 2 scheduling calls and 10 restaurant searches.
- **Checkpointer error.** An `AsyncPregelLoop` attribute error came from `checkpointer=False` on the agents combined with synchronous durability. They now inherit the parent checkpointer.

I also tested recovery by hand: I stopped the server during research and retried a failed plan after a restart, and the finished research was reused. I have not repeated this for the current revision.

9. Design decisions

- **Background runner.** The client polls or keeps an SSE connection open. The runner's lock covers one process, so more workers would need a shared job system.
- **SQLite for everything.** Drafts and review history sit next to the graph state. Startup recovery scans the checkpoints, which costs more as history grows.
- **Pydantic plus plain Python.** Pydantic checks the shape of each response, and Python computes totals. Gemini still picks places and estimates prices, so a response can pass validation while ignoring a dietary need, as the Lisbon dinner shows.

10. What I would do next

- Build an evaluation set: a single fado evening, a verified vegetarian meal, narrow budgets and closed attractions. Run it against different prompts and Gemini models.
- Improve the tools: OSRM for road routes, parsing of `opening_hours` strings, and a currency tool with a dated exchange rate.
- Prepare for more than one process: a queue, a shared checkpointer such as Postgres, and database locks.
- Add idempotency keys, authentication, per-user quotas, logs with `plan_id`, and a retention period for the SQLite checkpoints.

11. Assumptions

- The budget covers the whole group (lodging, food, activities and local transport). It excludes travel to the destination, including flights.
- Costs are in the request currency, and Gemini currently converts local prices.
- `TravelRequest` accepts trips of 1-14 days that start today or later, for 1-20 travellers.
- Prices, opening hours and events come from searches and Gemini, so travellers should confirm them before booking.